[width=8.0cm]WhatsApp Image 2026-10-10 at 11.24.10 PM.png

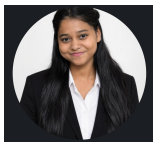
DSCPP-III-2026-T294

PROJECT AND TEAM INFORMATION

Project Title

Smart Water Management and Irrigation Optimization System Using C++

Student / Team Information

Team Name: *Team #*	*aquacode* *DSCPP-III-2026-T294*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Katiyar, Akanksha – 2510378334* *akankshakatiyar04@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Anand, Pragya – 2510011890* *anandpragya66@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Kumar Sahani, Shivendra – 2510370473* *shivendra7526048519sahani@gmail.com* 

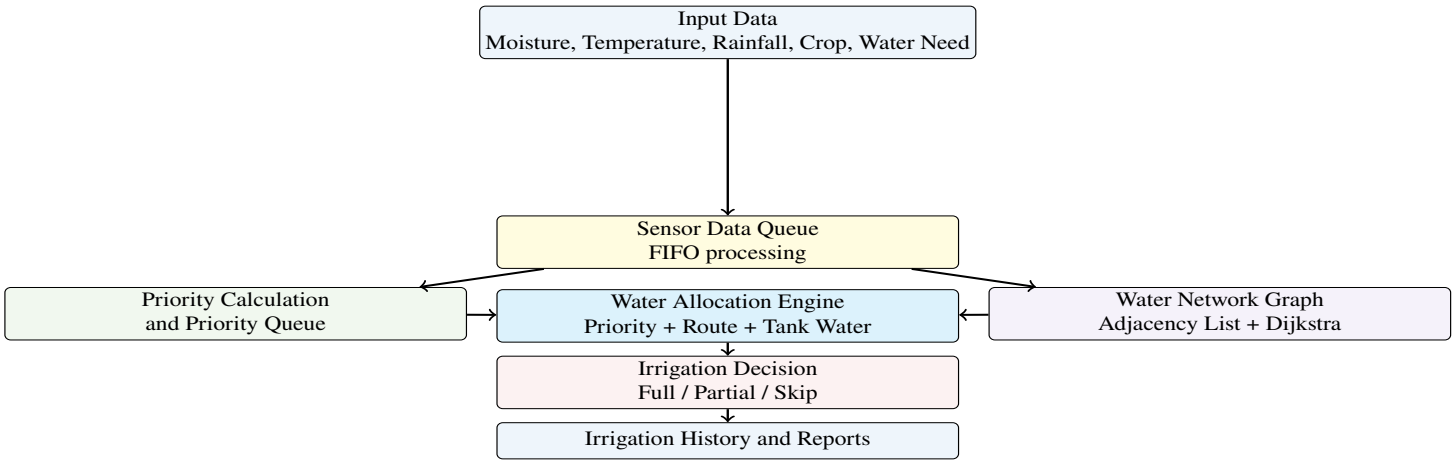
PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

- Our project, **Smart Water Management and Irrigation Optimization System**, is designed in C++ to efficiently manage water and reduce wastage. The system handles **100 farm zones** with information about crop type, soil moisture, temperature, probability of rainfall and water requirements.
- The sensor values can be entered manually or randomly generated for simulation.
- The sensor readings are managed in order by a **FIFO queue**, and the priority of irrigation is decided by soil moisture, temperature and rainfall.
- The project is a software simulation at this point and does not control physical sensors or water pumps.

Updated Project Approach and Architecture

- The system is a **menu-driven C++ application** that uses STL data structures to manage farm zones and irrigation.
- It manages **100 farm zones**. Sensor readings are entered manually or generated randomly and processed using a FIFO queue.
- Irrigation priority is calculated using soil moisture (50%), temperature (30%), and rainfall (20%). Zones with lower moisture, higher temperatures, and lower rainfall probability generally receive higher priority.
- A **priority queue** helps select zones for irrigation based on their priority.
- The water network contains **101 nodes**: one tank and 100 farm zones. Pipes are represented using an adjacency list, and Dijkstra's algorithm finds the shortest route from the tank to each zone.
- The graph supports adding, updating, and removing pipes. Before irrigation, the system checks water availability, rainfall, soil moisture, and route availability.
- After irrigation, the tank level, soil moisture, and remaining water requirement are updated. Irrigation history records the zone, water supplied, route, and pipe cost.
- The project uses standard C++ libraries such as `iostream`, `vector`, `queue`, `string`, and `iomanip`.
- The current system is a software simulation and does not operate physical sensors or water pumps.



Tasks Completed

Phase	Opted	Completed
Phase 1	1. Identified the problem and project objectives. 2. Planned DSA concepts and C++ implementation. 3. Developed the prototype and HTML dashboard.	**done**
Phase 2	1. Implemented the C++ code using DSA. 2. Developed and improved the HTML frontend. 3. Integrated, tested, and debugged the system.	**done**
Phase 3	1. Complete the working model. 2. Integrate the frontend with C++ code. 3. Test the system and fix errors.	*Ongoing*
Phase 4	1. Deploy the system in real farming conditions. 2. Integrate real sensors and water pumps. 3. Monitor irrigation and optimize water usage.	*Future*

Challenges/Roadblocks

- Integrating the farm, sensor, graph, and irrigation modules was challenging because all components needed to work together correctly.
- Calculating irrigation priority required considering soil moisture, temperature, and rainfall. A weighted formula was used to balance these factors.
- Implementing **Dijkstra's algorithm** and finding valid routes from the water tank to farm zones required careful testing.
- Limited water availability was another challenge. The system needed to decide when to provide full or partial irrigation or skip it.
- Handling invalid sensor readings, incorrect zone IDs, and disconnected pipes required additional input validation and error checking.
- These challenges were addressed by testing individual modules and then integrating them. Further edge-case testing and final validation are still in progress.

Tasks Pending

Task Pending
Final testing with more combinations of moisture, temperature and rainfall.
Testing additional pipe-network and disconnected-route cases.
More edge-case and invalid-input testing.
Verification of farm and sensor behaviour under different conditions.
Improvement of console-output formatting.
Final integration verification.
Final screenshots and documentation.
Final presentation and individual viva preparation.

Project Outcome/Deliverables

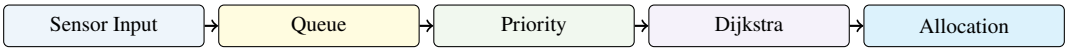
The project provides a working **C++ console-based Smart Water Management and Irrigation Optimization System**. It manages 100 farm zones and uses simulated environmental readings to make irrigation decisions.
The key deliverables are:

- 100 farm zones with crop and environmental information.
- Manual and random sensor-reading generation.
- FIFO sensor-data queue and priority calculation.
- Priority queue for selecting high-priority zones.
- Water-tank management and water addition.
- Rainfall and soil-moisture based irrigation decisions.
- Full and partial water allocation.
- Graph and adjacency-list based water network.
- Dijkstra's shortest-path algorithm.
- Pipe addition, updating and removal.
- Irrigation route, cost and history information.
- Menu-driven interface and irrigation summary.

Pump and valve operations are represented through console messages only. The current project does not control physical irrigation hardware.

Progress Overview

- The main Phase-II features have been implemented, including 100 farm zones, crop information, sensor simulation, FIFO queue processing, and irrigation-priority calculation.
- The water network uses an adjacency list with 101 nodes: one tank and 100 farm zones. Dijkstra's algorithm finds the shortest route, and pipes can be added, updated, or removed.
- The irrigation module checks priority, soil moisture, rainfall, tank-water availability, and route availability before allocating water.
- The system supports full or partial irrigation, updates water levels and zone data, and maintains irrigation history.
- The project is functionally complete for the current Phase-II stage. Remaining tasks include final testing, error checking, output improvements, screenshots, documentation, and presentation preparation.
- The current project is a software simulation and does not control physical sensors or water pumps.



Codebase Information

Repository: *Insert the team's actual GitHub repository link here.*
Branch: main
Programming Language: C++
Source-file ownership:

- `main.cpp` – **Akanksha**: main menu, program flow, module integration and irrigation/water-management control.
- `graph.cpp` – **Pragya**: graph, adjacency list, pipe management, Dijkstra, route reconstruction and network display.
- `farm.cpp` – **Shivendra**: farm-zone data, sensor readings, sensor queue, processing and priority calculation.

Main data structures: `Zone` and `Reading` structures, vector, queue, priority queue and adjacency-list graph.
Development stages: project setup; farm and sensor module; graph and Dijkstra module; irrigation/water-management integration; complete integration; testing and bug fixes.
The exact GitHub URL and actual commit IDs/messages should be inserted before final submission.

Testing and Validation Status

Test Type	Status	Notes
Farm-zone and crop creation	Pass	100 zones with crop data.
Sensor input and queue	Pass	Manual/random readings; FIFO processing.
Priority calculation	Pass	Uses moisture, temperature and rainfall.
Water tank management	Pass	Tracks available water.
Graph and Dijkstra	Pass	Finds shortest available routes.
Pipe management	Pass	Supports adding, updating and removing pipes.
Irrigation decisions	Pass	Checks rainfall, moisture and water availability.
Partial irrigation and history	Pass	Records water supplied and route details.
Module integration	Pass	Major modules work together.
Final edge-case testing	In Progress	Additional cases are being checked.

Deliverables Progress

Completed

- Designed the system and created 100 farm zones with crop data.
- Implemented sensor input and FIFO queue processing.
- Developed irrigation-priority calculation and priority queue.
- Implemented water-tank management and full/partial irrigation.
- Developed the graph, adjacency list, Dijkstra's algorithm and pipe management.
- Added irrigation decisions, history and menu-driven interface.
- Integrated the major project modules.

In Progress

- Testing sensor values, irrigation conditions and pipe networks.
- Improving console output and verifying module integration.
- Preparing screenshots and final documentation.

Pending

- Adding repository and commit details.
- Completing submission checks, presentation and viva preparation.

Overall Status: The main software features are implemented. Final testing, documentation and presentation preparation remain.

PROJECT STATUS

Core implementation completed | Testing and documentation in progress | Final presentation pending